

Overview of Statistical Learning and Modeling – 36-600

Lab 4T: Unsupervised Learning I
Due Wednesday, September 17, 11:59pm

Trishla Nair

To answer the questions below, it will help you to refer to Sections 10.3 and 10.5 of the ISLR book (the James et al. book listed on the syllabus); it might also help you to refer to previous labs.

Question 1

Let's create a fake data frame with three rows and three columns:

```
df <- data.frame(x = 1:3, y = 1:3, z = 1:3)
ed_1_2 <- sqrt(1 + 1 + 1) #sqrt(((2-1)^2)*3)
ed_1_3 <- sqrt(4 + 4 + 4) #sqrt(((3-1)^2)*3)
ed_2_3 <- sqrt(1 + 1 + 1) #sqrt(((3-2)^2)*3)
print(ed_1_2, ed_1_3, ed_2_3)
```

```
## [1] 1.73
```

By hand, compute what is the Euclidean distances between the fake datum of row 1 and the fake data of rows 2 and 3? And what is the Euclidean distance between the fake data of rows 2 and 3?

The euclidean distance between row 1 and 2 is $\sqrt{3}$. the distance between 1 and 3 is $2\sqrt{3}$. The di

Question 2

Now compute the Euclidean distances using the `dist()` function. Show the output from this function. Does that output match your hand-computed quantities? If not, go back and think re-do your calculation in Question 1. If so, then you now have a sense as to what the “distance between two data points” is, in practice. Note the appearance of the output: the distances are stored in a lower-triangular matrix.

```
dist(df)
```

```
##           1           2
## 2 1.732051
## 3 3.464102 1.732051
```

Dataset 1

Here we import some data on stars which are either in, or in the same general direction as, the Draco Dwarf Galaxy.

```
file.path <- "https://github.com/FSKoerner/F25-36600-data/raw/refs/heads/main/draco_photometry.Rdata"
load(url(file.path))
df <- data.frame(ra, dec, "v" = velocity.los, log.g,
                 "g" = mag.g, "r" = mag.r, "i" = mag.i)
rm(file.path, ra, dec, velocity.los, log.g, temperature,
    mag.u, mag.g, mag.r, mag.i, mag.z, metallicity, signal.noise)
head(df)
```

```
##      ra      dec      v log.g      g      r      i
## 1 260.0115 57.81589 -294.1   3.7 19.70348 19.07734 18.81921
## 2 259.8737 57.71628 -205.1   3.1 20.11101 19.43237 19.13829
## 3 259.9352 57.79092 -282.1   4.5 20.26365 19.66915 19.40216
## 4 259.9352 57.79092 -283.7   2.1 20.26365 19.66915 19.40216
## 5 260.0805 57.85542 -277.5   1.2 20.05116 19.47914 19.16447
## 6 260.0039 57.75311 -298.2   3.7 20.33827 19.71865 19.45009
```

df is a data frame with 2778 rows and 7 columns. See this [README file](#) for a full description of the data and its variables.

Question 3

Use `dplyr` functions to filter the data frame such that it only contains values of `dec > 56`, values of `ra < 264`, and values of `v` between -350 and -250. (**Important:** the `tidyverse` style guide says to separate comparison operators, like `<`, from the objects they are comparing with a space. I'll always try to use proper style in assignments and solutions, and you should to, but it won't always be clear what tangible benefit that confers. In this case, you'll absolutely need a space between the `<` and the `-` when you filter on the latter velocity, otherwise R will see the assignment operator and act accordingly.) Mutate the data frame to have `g-r` and `r-i` colors (call them `gr` and `ri`), then delete the magnitudes and `v`. (*Recall:* you can “negatively select” columns by putting minus signs in front of the column names.) Save the resulting data frame as `df.new`.

```
suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
df.new <- df %>% filter(dec > 56, ra < 264, -350 < v, v < -250)
df.new <- df.new %>% mutate(gr = g-r) %>% mutate(ri = r-i)
df.new <- df.new %>% select(-v, -g, -r, -i)
head(df.new)
```

```
##      ra      dec log.g      gr      ri
## 1 260.0115 57.81589   3.7 0.6261406 0.2581272
## 2 259.9352 57.79092   4.5 0.5945015 0.2669868
## 3 259.9352 57.79092   2.1 0.5945015 0.2669868
## 4 260.0805 57.85542   1.2 0.5720177 0.3146687
## 5 260.0039 57.75311   3.7 0.6196175 0.2685547
## 6 260.0039 57.75311   1.2 0.6196175 0.2685547
```

Question 4

Use the `kmeans()` function to cluster the data in your data frame. Try different values for k , and finally display results for what *you* would choose as its optimal value. The default for `nstart` is 1; you'll probably want to increase that to something larger – play with the values for this argument.

Install the package `GGally` and uncomment the library-loading line below. Then display your results using the `ggpairs()` function; for example, use

```
ggpairs(df.new, progress = FALSE, mapping = aes(color = factor(km2.out$cluster)))
```

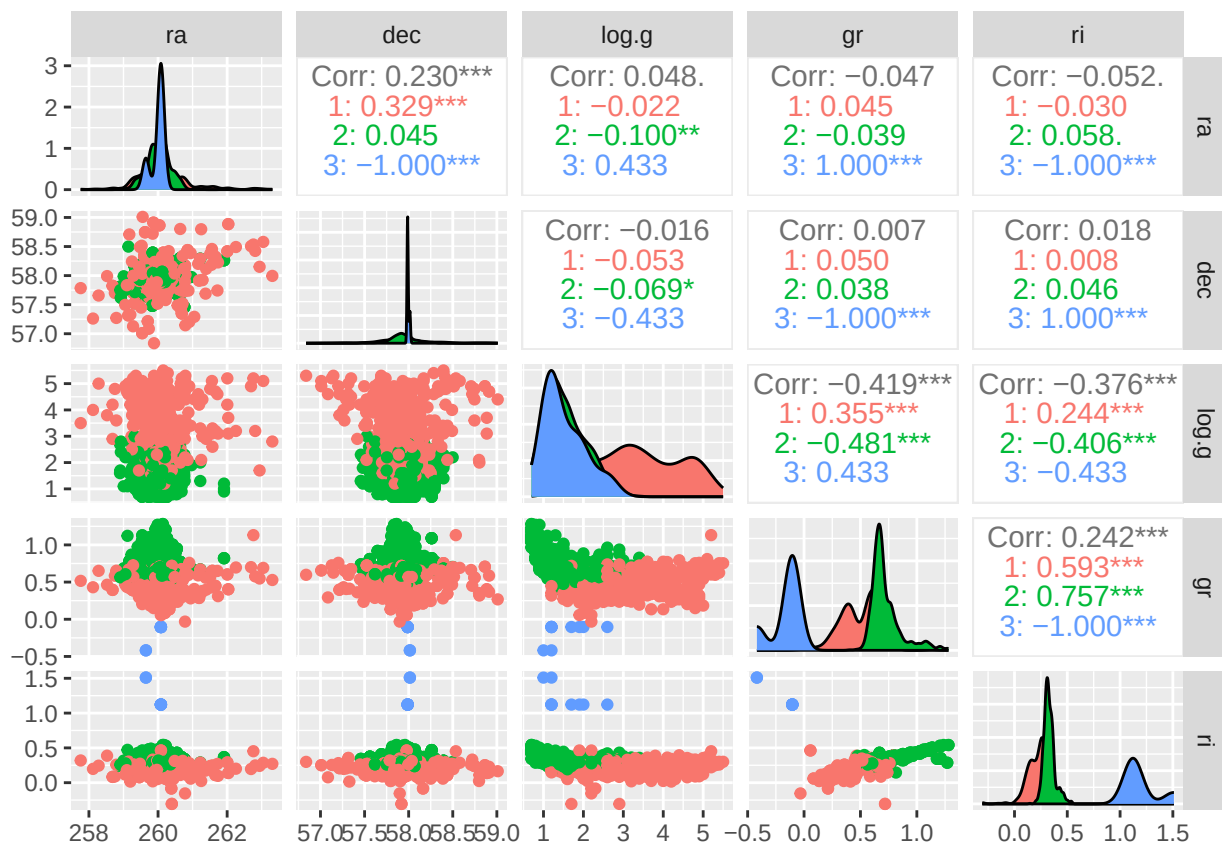
where `km2.out` is what I call the output from `kmeans()` for $k = 2$.

Remember to `scale()` your data when you call `kmeans()` (but not when plotting)! Also, note that `kmeans()` uses random sampling, so you should absolutely set a random number seed immediately before calling `kmeans()` to ensure reproducibility.

```
suppressMessages(library(GGally))
```

```
## Warning: package 'GGally' was built under R version 4.4.3
```

```
set.seed(101)
km3.out <- kmeans(scale(df.new), 3, nstart = 20)
ggpairs(df.new, progress = FALSE, mapping = aes(color = factor(km3.out$cluster)))
```



```
# FILL ME IN
```

Question 5

For your chosen k value, what are the number of groups and the number of data in each group? Also, what is ratio of the between-cluster sum-of-squares to the total sum-of-squares? (This is a measure of the total variance in the data that is “explained” by clustering. Higher values [closer to 100%] are better, but beware: the larger the value of k , the higher the ratio is going to be: you will be getting into the realm of *overfitting*.) (Hint: `print()` your saved output from `kmeans()`.)

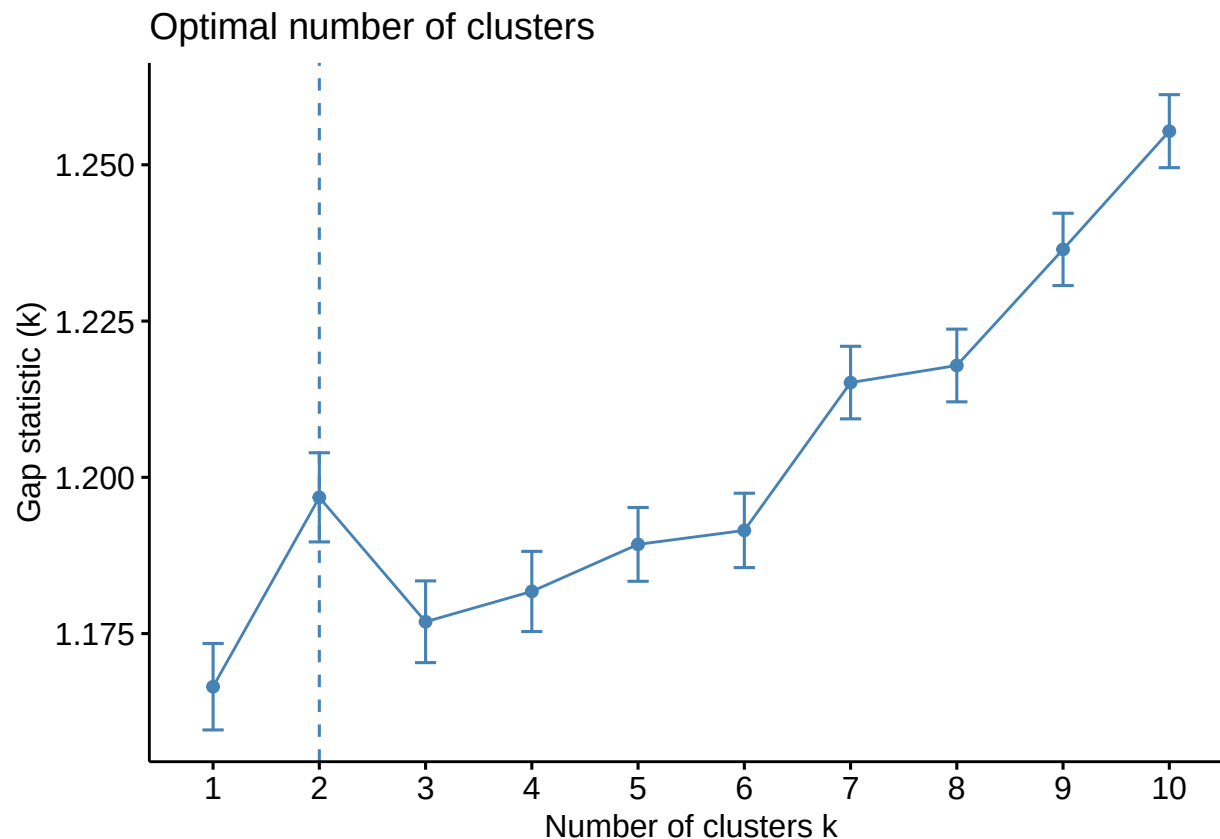
```
print(km3.out)
```

```
## K-means clustering with 3 clusters of sizes 348, 860, 10
##
## Cluster means:
##          ra          dec        log.g          gr          ri
## 1  0.16401832  0.04319432  1.2190653 -0.7956966 -0.7430474
## 2 -0.06499155 -0.02057814 -0.4872214  0.3718246  0.2130048
## 3 -0.11856394  0.26655810 -0.5224333 -4.2866784  7.5396351
##
## Clustering vector:
##  [1] 1 1 2 2 1 2 1 2 2 2 2 2 2 2 2 1 2 2 1 2 2 2 2 2 2 1 2 1 2 2 1 2 2 2 2 2
## [38] 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [75] 1 1 1 1 1 2 2 1 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 1 2 1 2 1 2 2 2 2 1 2 2
## [112] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [149] 2 2 2 3 3 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2
## [186] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 1 2 2
## [223] 2 2 2 2 2 1 2 2 1 2 1 2 2 2 1 2 2 2 2 2 2 2 1 2 1 2 2 2 2 2 2 2 2 2 2 2
## [260] 2 1 2 2 2 2 1 1 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [297] 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 2 2 1 2 2 2
## [334] 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 2 2
## [371] 1 1 2 2 2 2 2 2 2 2 2 2 2 1 2 1 1 2 2 2 2 2 1 2 1 2 2 2 1 2 2 2 2 2 1 1 1
## [408] 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 2 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [445] 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 2 2 2 1 2 2 2 2 2 2 2 2 2
## [482] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1
## [519] 2 2 2 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 1 1
## [556] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 2 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 1 2
## [593] 2 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [630] 2 2 2 1 2 1 2 2 1 2 2 1 1 1 1 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2 1 2 2 2 2 2
## [667] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [704] 2 2 2 2 1 2 2 1 2 2 2 2 2 2 2 2 2 2 2 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2
## [741] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 2 1 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2
## [778] 2 2 1 1 1 2 2 2 2 2 2 2 2 2 2 2 1 1 2 2 2 2 2 2 2 2 2 2 2 1 1 2 2 1 2 1 2
## [815] 1 2 2 2 2 2 2 2 2 2 2 2 2 1 2 2 1 1 1 1 2 2 1 2 1 1 1 2 2 2 1 1 2 2 2 2
## [852] 2 2 2 2 2 2 2 2 2 1 1 1 1 2 2 2 2 2 1 1 2 2 2 2 2 2 2 2 2 1 1 2 2 2 1 1 2
## [889] 1 1 2 2 1 2 2 2 2 1 1 1 1 1 1 1 2 2 1 1 1 2 1 1 1 2 2 2 1 2 1 1 1 1 1 1
## [926] 1 1 1 1 1 2 1 1 1 1 1 2 2 1 2 1 1 1 1 1 1 1 1 2 2 2 2 1 2 2 2 2 2 1 2 1
## [963] 1 1 1 1 1 1 2 2 2 2 1 1 1 1 1 1 1 1 1 2 2 2 2 1 1 2 3 3 3 3 3 3 3 1 1 1
## [1000] 2 2 1 2 2 1 1 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 2 2 1 1 1 2 2 2 2 1 2 1 1
## [1037] 1 1 2 2 2 1 1 1 2 1 1 2 2 1 2 1 1 1 2 1 2 1 1 2 2 2 1 2 1 1 2 2 2 1 1 2 2
## [1074] 2 1 1 1 1 1 1 1 2 2 1 2 2 2 2 2 1 2 1 2 2 2 1 2 2 2 1 1 1 1 1 2 1 1 1 1
## [1111] 1 2 2 1 2 2 2 2 1 1 2 1 2 2 2 2 1 2 2 2 2 1 1 2 2 1 1 1 1 1 1 1 1 1 1 1
```

I have 3 clusters of sizes 348, 860, 10. Within cluster sum of squares by cluster values are 2347.98887

Now use one of the methods presented in class for determining the optimal value of k , while noting that some methods provide more concrete results than others.

```
## Warning: 'aes_string()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'.
## i See also 'vignette("ggplot2-in-packages")' for more information.
## i The deprecated feature was likely used in the factoextra package.
##   Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



Dataset 2

Question 7

Now read in the diamonds dataset (`diamonds.csv`); download it from Canvas. Once the data frame is input (be sure to use the argument `stringsAsFactors = TRUE`, by the way), remove the columns `X`, `cut`, `color`, and `clarity` from it, and call the resulting data frame `df.new`.

```
df<-read.csv("diamonds.csv", stringsAsFactors = TRUE)
df.new<-df%>%select(-X, -cut, -color, -clarity)
head(df.new)
```

```
##   carat depth table price    x    y    z
## 1  0.23  61.5   55  326 3.95 3.98 2.43
## 2  0.21  59.8   61  326 3.89 3.84 2.31
## 3  0.23  56.9   65  327 4.05 4.07 2.31
## 4  0.29  62.4   58  334 4.20 4.23 2.63
## 5  0.31  63.3   58  335 4.34 4.35 2.75
## 6  0.24  62.8   57  336 3.94 3.96 2.48
```

Question 8

The diamonds dataset is too large to input into *k*-means as is. So we will randomly select 1000 rows from the data frame and work with those. First, run this code (while noting that you can change the seed to

whatever you like):

```
set.seed(303)
s <- sort(sample(nrow(df.new), 1000))
```

Then use the `slice()` function from `dplyr` to select the rows recorded in the vector `s`, and save the output to `df.small`.

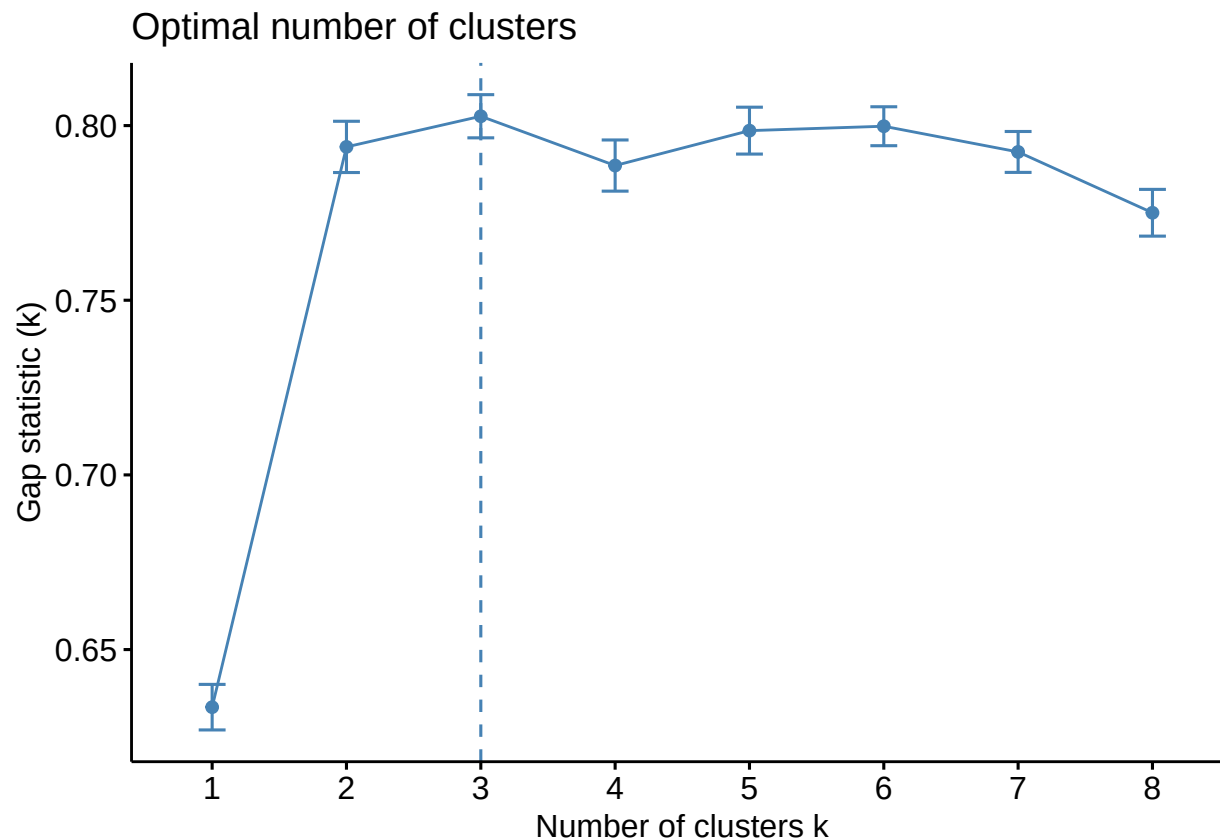
```
set.seed(303)
s <- sort(sample(nrow(df.new), 1000))
df.small <- df.new %>% slice(s)
head(df.small)
```

```
##   carat depth table price     x     y     z
## 1  0.30  62.6    59   552  4.23  4.27  2.66
## 2  0.26  62.6    59   554  4.06  4.09  2.55
## 3  0.70  62.7    55  2774  5.68  5.74  3.58
## 4  0.53  57.5    64  2782  5.34  5.37  3.08
## 5  0.55  60.9    57  2789  5.28  5.30  3.22
## 6  0.70  60.6    58  2822  5.80  5.72  3.49
```

Question 9

Now we can do *k*-means. But we'll do things a little differently this time: here, just run `clusGap()` directly with `K.max = 8` and plot the output with `fviz_gap_stat()`. Keep that value in mind for the next question below.

```
cg <- clusGap(scale(df.small), FUN = kmeans, nstart = 20, K.max = 8, B = 50)
fviz_gap_stat(cg)
```

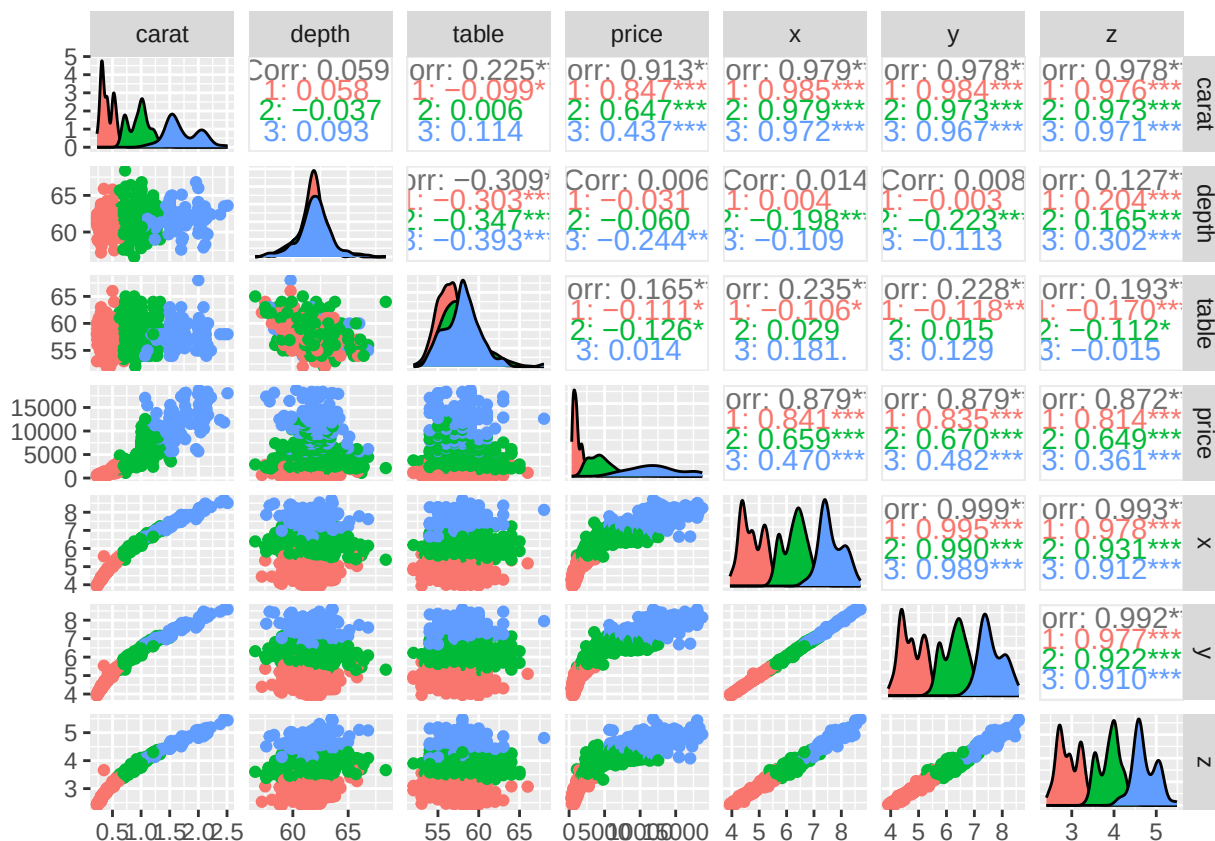


Question 10

In Question 4, we explored using different values of k , and through the use of `ggpairs()` we determined that for the stellar data, k might reasonably be 2 or 3. In Question 9, on the other hand, we let the gap statistic function determine the optimal value for k – and now here we will use `ggpairs()` to visualize the results of assuming this optimal value only. Do this below.

You need not state an answer to this question, but: does it appear that there are natural clusters in the data that were uncovered using k -means, or does it appear that the data were simply split into pieces? (Look at, e.g., `price` versus `carat`; that scatterplot *might* influence your thinking. Note that there not necessarily a right answer here. Interpretation, meet ambiguity.)

```
km3.out <- kmeans(scale(df.small), centers= 3, nstart = 20)
ggpairs(df.small, progress = FALSE, mapping = aes(color = factor(km3.out$cluster)))
```



Question 11

Read in the file `diamonds.csv` again, using `read.csv()` with the argument `stringsAsFactors = TRUE`, but do *not* remove any columns. However, do select a subset of 1000 rows again. Here we will examine these data with the k -prototypes algorithm.

Install the package `clustMixType`, then load it into R via a call to the `library()` function (uncomment the line below). Then run the function `validation_kproto()`. (To figure out how to run it, go to the help page for the function and scroll down to the examples. Use the silhouette method, set the data to `df.small` [your 1000-row subset of `df`], set `k` to 2:5, and set `nstart` to 5. Add one more argument: `verbose = FALSE`. Do not save your output to a variable – just let `validation_kproto()` output directly to your screen.) Examine the output. `indices` is the silhouette score for each value of k . What is the optimal number of clusters?

```
library(clustMixType)
```

```
## Warning: package 'clustMixType' was built under R version 4.4.3
```

```
df<-read.csv("diamonds.csv", stringsAsFactors = TRUE)
set.seed(202)
s <- sort(sample(nrow(df), 1000))
df.small<-df[s,]

print(validation_kproto(method = "silhouette", data = df.small, k = 2:5, nstart = 5, verbose = FALSE))
```

```
## $k_opt
## [1] 2
##
## $index_opt
##      2
## 0.7095233
##
## $indices
##      2      3      4      5
## 0.7095233 0.5510503 0.5078248 0.4301635
##
## $kp_obj
## Distance type: huang
##
## Numeric predictors: 8
## Categorical predictors: 3
## Lambda: 40548786
##
## Number of Clusters: 2
## Cluster sizes: 478 522
## Within cluster error: 66507189684 89230726681
##
## Cluster prototypes:
##      X      carat  cut color clarity  depth  table  price      x
## 1 40502.67 0.4609623 Ideal    E    VS2 61.68682 57.29456 1276.402 4.907762
## 2 13768.59 1.1030651 Ideal    H    SI1 61.75287 57.83295 6425.331 6.478123
##      y      z
## 1 4.913619 3.029059
## 2 6.476360 3.999080
##
## $type
## [1] "huang"
```

```
# FILL ME IN
```

The optimal number of clusters appear to be 2.

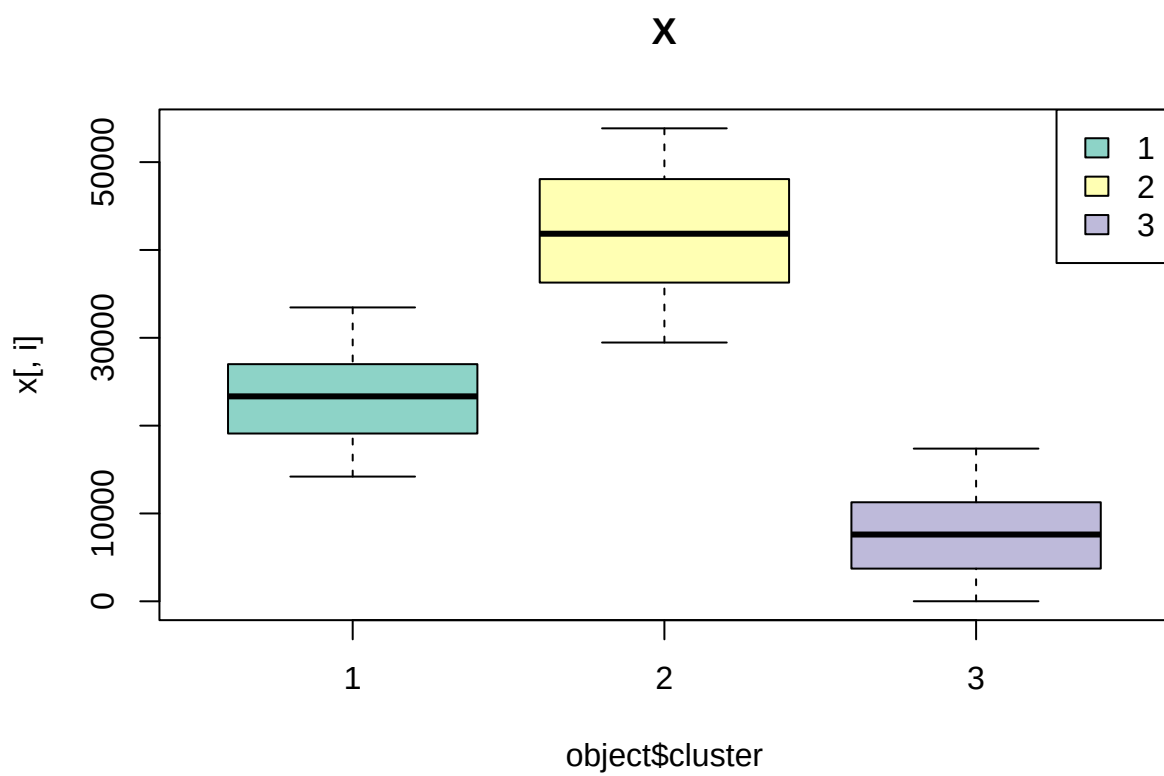
Question 12

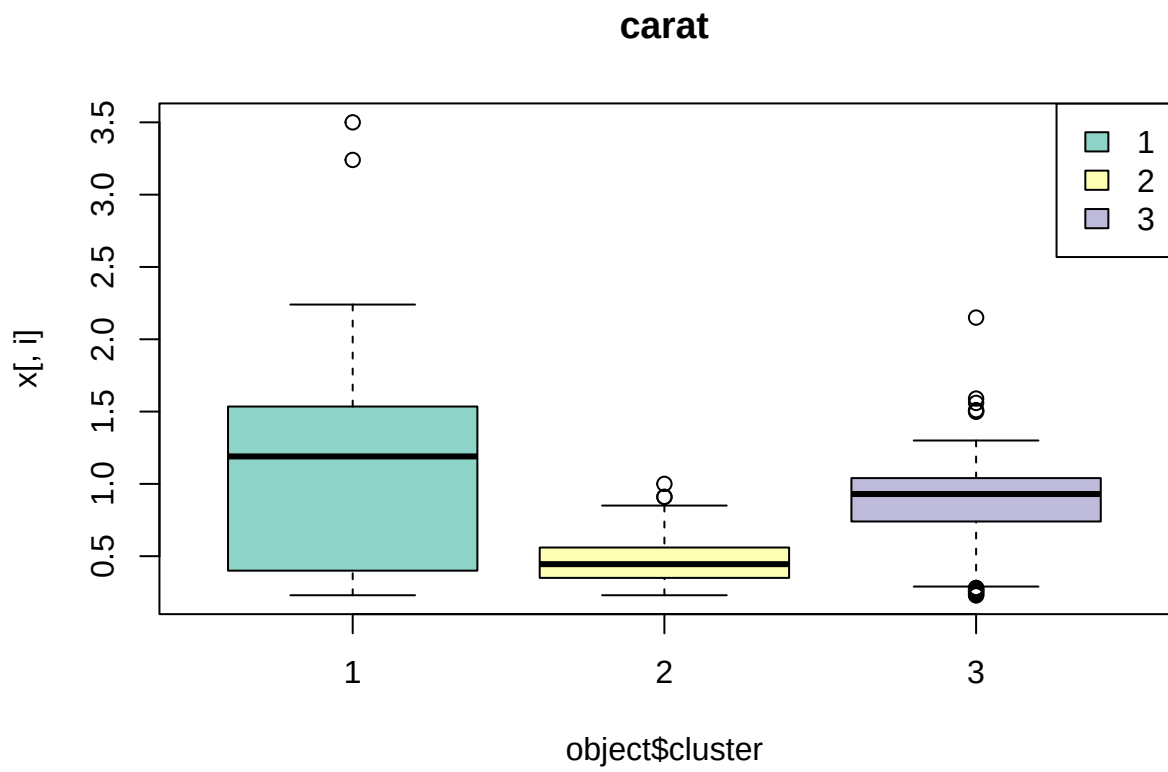
As one last exercise, run

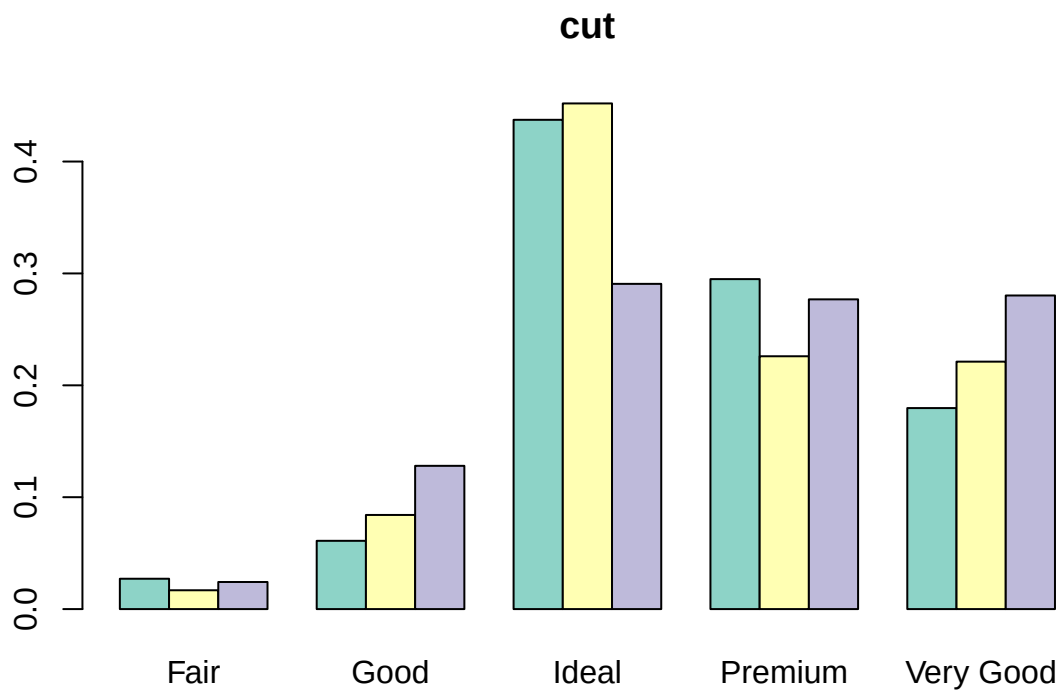
```
kp.out <- kproto(df.small, <# OF CLUSTERS>, nstart = 5, verbose = FALSE)
```

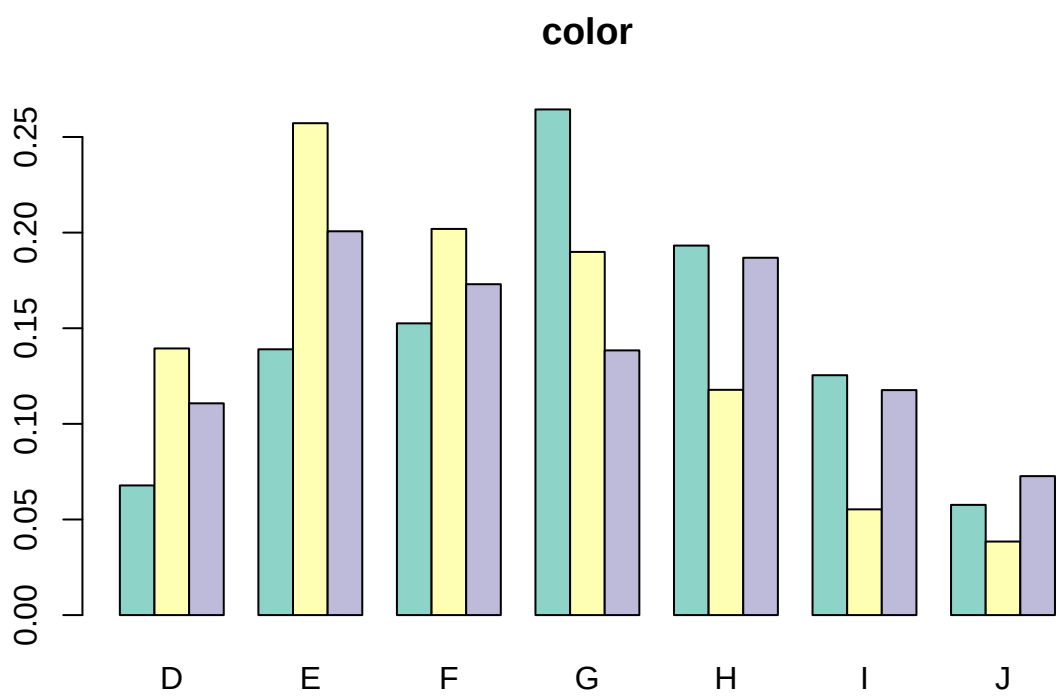
and then plot the output: `plot(kp.out)`. You may want to knit the file and then look at the plots at the end of the output, rather than running this code chunk separately. You need not comment on these plots, but you should realize that they show the values of each variable for each cluster, and thus they would help you construct a story about how the clusters differ. For instance, I see that the X variable values differ greatly between clusters, although similar effects are apparent for a few variables.

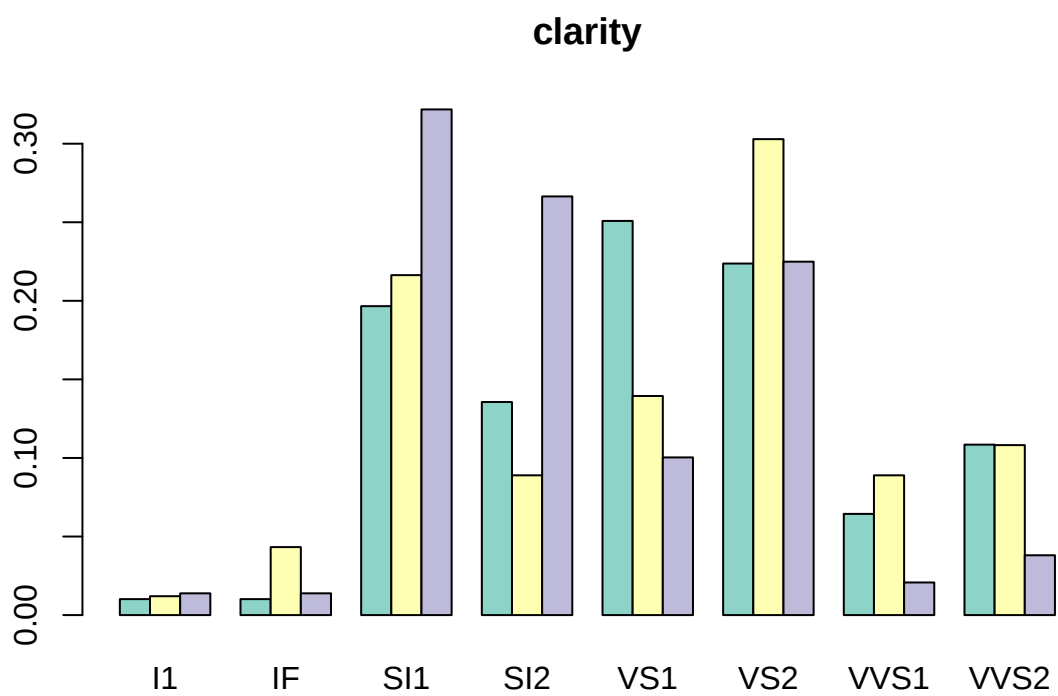
```
kp.out <- kproto(df.small, 3, nstart = 5, verbose = FALSE)
plot(kp.out)
```

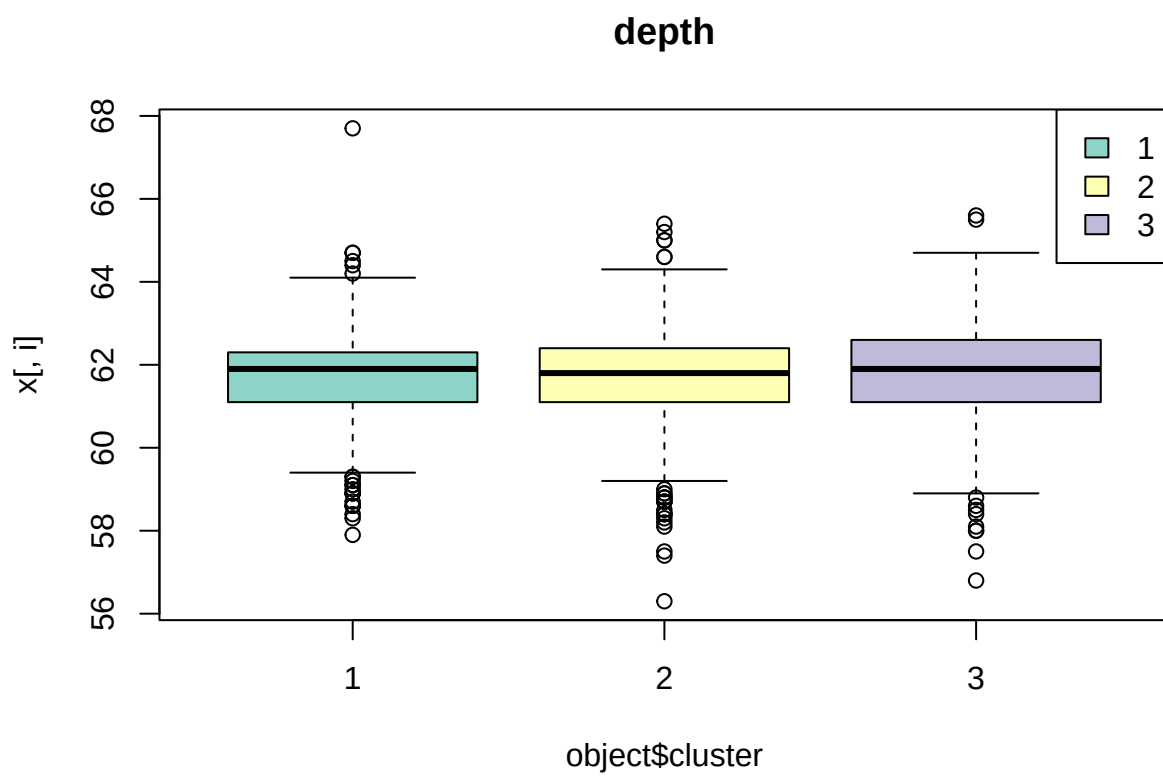












table

